

# Taledge Enterprise AI Interview Pipeline: Deep Architecture

## Confidential & Proprietary

This whitepaper details the exact sequence diagrams, payload schemas, asynchronous queuing, and sub-system architectures that power the Taledge Dual-Track AI Interview Engine. The system leverages an asynchronous, bi-directional streaming architecture with real-time semantic processing, advanced adversarial LLM routing, and heuristic proctoring matrices to guarantee zero-hallucination cognitive evaluations.

## 1. SYSTEM INITIALIZATION & DETERMINISTIC STATE HYDRATION

Before the WebRTC or WebSocket layers establish a connection, the **EngineController** hydrates a strict execution context. This prevents model hallucination and binds the Large Language Model (LLM) to a deterministic evaluation path. It prevents prompt injection from malicious candidates by locking the semantic bounds of the conversation entirely.

### 1.1 The Multi-Dimensional Context Payload

The frontend compiles an aggregated, token-optimized JSON structure containing parsed ATS data, Job Description (JD) Vector Embeddings, and Academic historical markers.

```
{  
  "session_configuration": {
```

```
    "session_id": "vtx_99812a_b7",
    "jwt_session_token":
"eyJhbGciOiJIUzI1NiIsInR5c...",
    "execution_mode": "adversarial_tech_stage_1",
    "latency_tolerance_ms": 1500,
    "cognitive_load_multiplier": 1.5,
    "fallback_llm_cluster": "cluster_beta_eu"
},
"candidate_matrix": {
    "identity_hash": "2f4b9d0a",
    "target_role": "Senior Cloud Solutions
Architect",
    "skill_graph_embeddings": [0.012, -0.045,
0.992, -0.114, 0.544],
    "verified_tech_stack": {
        "primary": ["Kubernetes", "AWS EKS", "Go",
"gRPC"],
        "secondary": ["PostgreSQL", "Terraform",
"Redis"]
    },
    "impact_claims": [
        {
            "domain": "Distributed Systems",
            "claim": "Engineered a zero-downtime
deployment pipeline scaling to 50k RPS.",
            "validation_flag": "REQUIRES_DEEP_PROBING"
        }
    ]
}
```

## 1.2 Redis Semantic Caching & State Management

---

During initialization, the payload is committed to a highly available Redis cluster (running in a multi-AZ active-active deployment). As the interview progresses, the entire conversational turn history is pushed into a Redis List structure. This allows the API stateless workers to instantly rebuild the conversational context upon every new chunk of transcribed speech, reducing database lookup latency to sub-2ms bounds.

## 2. THE DETERMINISTIC LLM PERSONA ROUTING

---

The "Brain" of the system relies on dynamic system-prompt injection.

Depending on the **execution\_mode**, the context is routed to specialized sub-models. The system operates on a "Chain of Personas" architecture.

### 2.1 The Elite Technical Adversary (Stage 1)

---

Instead of a generic conversational system prompt, the LLM is bound by a strict adversarial framework designed to induce, track, and measure cognitive load.

```
SYSTEM PROMPT BINDING - KERNEL LAYER:
[Role]: Elite Principal Engineer & Hostile System Architect.
[Language Parameters]: Multilingual continuous (English/Hindi/Hinglish) with auto-detection.

[Execution Directives]:
1. TARGET CLAIMS: Isolate the candidate's claim:
"zero-downtime deployment pipeline scaling to 50k
```

RPS."

2. **STRESS VECTOR:** Ask them how their architecture handles a split-brain network partition in the Redis caching layer during a deploy.

3. **COGNITIVE LOAD:** If they answer correctly, immediately compound the load by introducing a sudden IOPS bottleneck on the primary PostgreSQL shard. Do not let them retreat to safe topics.

4. **OUTPUT FORMAT:** Maximum 45 words. High density. No pleasantries. Probe edge cases strictly. Do not validate their answers.

5. **FALLBACK ROUTING:** If candidate refuses to answer, terminate thread and shift to {verified\_tech\_stack.secondary}.

## 2.2 The Psychometric DNLA Analyst (Stage 2)

In the behavioural phase, the system fundamentally shifts its semantic evaluation model from technical accuracy to psychological structural integrity.

### SYSTEM PROMPT BINDING - BEHAVIOURAL LAYER:

[Role]: Clinical Behavioural Psychologist & Enterprise HR Director.

[Execution Directives]:

1. **BEHAVIOURAL TRAPPING:** Bypass standard STAR (Situation, Task, Action, Result) methodology. Induce cognitive dissonance by asking them to defend a time they knowingly bypassed protocol to

meet a deadline.

2. **LINGUISTIC MAPPING:** Track Pronoun Indexing (I vs We). If 'We' is used for success and 'They' for failure, trigger the [Accountability Deficit] flag.

3. **EMOTIONAL REGULATION:** Apply conversational pressure. Challenge their narrative to measure baseline defensive mechanism triggers. If defensive language is detected (e.g. "Like I already said"), trigger [Low Resilience] flag.

### 3. THE BI-DIRECTIONAL STREAMING PIPELINE

---

The conversational loop is not a standard HTTP Request/Response model. Such a model introduces unacceptable latency. The Taledge architecture uses an ultra-low latency, bi-directional asynchronous stream.

#### 3.1 Step 1: Auditory Ingestion & VAD Buffering

---

The candidate speaks into the microphone. The browser captures audio chunks via the **MediaRecorder API**, feeding it into a local Voice Activity Detection (VAD) buffer.

- The VAD algorithm monitors dB thresholds to separate silence from speech.
- Once a pause of **700ms** is detected, the transcribing Webkit SpeechRecognition API fires the finalized text payload.
- This mitigates the "interrupt" problem where the AI responds before the candidate finishes a thought.

#### 3.2 Step 2: Pre-Processing & Semantic Sanitization

---

Before hitting the LLM, the raw text string is passed through a lightweight Edge Function normalizer.

- Filler words ("um", "uh", "like") are stripped.
- Core intents are extracted.
- This process reduces token bloat by roughly 18%, significantly lowering inference costs and accelerating the LLM's Time-To-First-Token (TTFT).

### 3.3 Step 3: LLM Inference Generation & OpenRouter Fallback Clusters

---

The backend streams the normalized history to the LLM router network. The generation is strictly parameterized:

- **temperature: 0.6** : Prevents wild hallucinations while maintaining conversational fluidity.
- **top\_p: 0.8** : Restricts the sampling pool to highly probable logical tokens.
- **The Fallback Chain**: If the primary model endpoint times out (latency > 1500ms), the OpenRouter integration instantly reroutes the payload to a secondary cluster without dropping the connection, ensuring zero downtime during the interview.

### 3.4 Step 4: Chunked Server-Sent Events (SSE) & Real-Time TTS

---

To eliminate waiting latency, the architecture relies on chunked generation.

- The LLM's output stream is intercepted by the backend.
- The backend chunks the stream by sentence delimiters ( . , ? , ! ).
- As soon as the first sentence is completely generated, it is pushed to the frontend client via Server-Sent Events (SSE).
- The frontend immediately vocalizes the sentence using the browser's **SpeechSynthesis** Web API.
- While the first sentence is being spoken, the LLM continues generating the second and third sentences in the background, achieving theoretical zero-latency conversational dynamics.

## 4. MULTI-VECTOR SECURITY, PROCTORING & INTEGRITY WRAPPERS

---

To guarantee assessment integrity, the interview environment does not rely merely on trust. It runs inside a heavily sandboxed, heuristic monitoring wrapper that aggressively flags anomalous behavior.

#### 4.1 Heuristic State Monitoring

---

- **Context-Switch Deterrence:** A strict `visibilitychange` listener is bound to the document root. If `document.hidden` returns `true` (meaning the candidate switched tabs to Google an answer), a Level 1 Violation is recorded. At 3 violations, the WebSocket is forcefully severed, the session is burned, and an immutable cheating flag is written to the database.
- **Copy-Paste Nullification:** All standard clipboard events ( `keydown` for `Meta+C/V` , `contextmenu` ) are intercepted at the window level, and `preventDefault()` is called.

#### 4.2 Micro-Expression Latency Tracking

---

One of the most advanced features of the pipeline is its ability to measure cognitive hesitation.

- The system calculates the exact millisecond delta between the AI finishing its speech ( `onend` event of `SpeechSynthesis`) and the candidate beginning theirs (VAD trigger).
- If the latency consistently exceeds 4000ms, it indicates deep conceptual searching or potential screen-reading.
- This metric is compiled into the `High_Hesitation_Index` and passed directly to the Scoring Algorithm, heavily weighting the candidate's final confidence score downwards.

## 5. NETWORK PROTOCOL & INFRASTRUCTURE

---

#### 5.1 WebSockets vs. WebRTC

---

While WebRTC is traditionally used for audio/video streaming, the Taledge AI pipeline relies on **Secure WebSockets (WSS)** for the primary data transmission layer.

- Because the audio is transcribed locally on the client using the browser's native engine, we are transmitting extremely lightweight JSON text payloads rather than heavy audio buffers.

- This reduces bandwidth requirements by 99%, allowing candidates on 3G cellular connections in remote areas to participate in the interview flawlessly without packet loss.

## 5.2 Zero-Trust Authentication

---

Every payload sent over the WebSocket is verified using a short-lived JSON Web Token (JWT).

- Tokens are minted at session initialization and have a strict expiry time matching the `maxDuration` of the interview (e.g., 30 minutes).
- If a candidate attempts to intercept the WebSocket traffic and inject a forged STT payload, the API gateway immediately rejects the un-signed payload and terminates the socket connection.

## 6. SESSION TERMINATION & HIGH-THROUGHPUT POST-PROCESSING

---

Upon termination (either via natural token limit exhaustion, time expiry, or a security breach), the `EngineController` compiles the entire transcript, the latency metadata, and the proctoring logs into a finalized `AssessmentLedger`.

```
{
  "ledger_id": "ldg_99812a_b7",
  "completion_status": "NATURAL_EXHAUST",
  "total_latency_variance_ms": 340.5,
  "proctoring_flags": 0,
  "transcript": [
    {"role": "assistant", "content": "How do you handle a split-brain Redis partition?"},
    {"role": "user", "content": "I would implement a quorum-based sentinel architecture..."}
  ]
}
```



```
}
}
```

This ledger is then placed into a high-throughput message queue (Apache Kafka). The asynchronous Scoring Engine consumes this ledger, mapping the thousands of data points against the 5-Vector Algorithmic Matrix to produce the final, mathematically indisputable **Success Potential Score**.